

Image to Text Dataset for Quantum Computing

Ammar Haider, a.haider2@oth-aw.de, ExamID:36^{1,†}

¹OTH Amberg-Weiden, Study programme "Master of Artificial Intelligence for Industrial Applications"

Abstract

Recent advances in deep learning have increased demand for large-scale, domain-specific datasets. In quantum computing, obtaining high-quality datasets of quantum circuit diagrams paired with textual descriptions remains challenging due to the complexity of extracting structured visual content from scientific literature. This work presents an automated **render-based pipeline** for constructing a high-fidelity image-text dataset of quantum circuits from arXiv publications. The pipeline operates directly on $\text{\LaTeX}$ source code, identifying quantum circuit environments (quantikz, qcircuit), extracting gate-level metadata through multi-stage parsing and normalization, and recompiling circuits into standalone images at 300 DPI resolution. This source-level approach achieves zero false positives and guarantees structural correctness, avoiding the ambiguity and noise inherent in PDF-based extraction methods. Applied to a corpus of 26907 quantum computing papers from arXiv, the pipeline successfully renders 250+ circuits using only 1917 papers from this corpus with comprehensive metadata including gate sequences, quantum problem classifications, page numbers, and contextual descriptions. The resulting dataset provides an ideal foundation for fine-tuning Image to text models in the quantum computing domain, with potential applications in automated circuit analysis, quantum algorithm recognition, and educational tooling. A prototypical text-driven discovery approach for capturing visually diverse circuit representations from PDF is also discussed briefly as future work to supplement the core render-based dataset.

Keywords: Natural Language Processing, Information Retrieval, Lexical Semantics, TFIDF, SentenceBERT, Quantum Circuits, Arxiv API, Latex Extraction

1. Introduction

Recent progress in computer vision and multi-modal learning has demonstrated strong performance on image-text tasks when high-quality visual data is paired with accurate textual descriptions [1]. However, specialized domains such as quantum computing lack standardized image-text datasets, creating a significant gap for multi-modal learning applications in this field.

Extracting quantum circuit images from scientific documents presents unique challenges. Circuits are commonly defined using specialized $\text{\LaTeX}$ environments such as quantikz or qcircuit, often embedded within complex documents containing mathematical expressions, tables, and layout elements. PDF-based extraction methods frequently introduce false positives and struggle to reliably isolate circuit figures from surrounding content.

This project presents an automated **render-based pipeline** for extracting quantum circuit images and metadata from arXiv papers. The pipeline directly recompiles $\text{\LaTeX}$ circuit environments into standalone images, achieving an empirically observed false positive rate of zero and perfect structural correctness. By operating directly on circuit source code rather than compiled PDFs, the system guarantees accurate representation of quantum gates, circuit topology, and computational structure.

The pipeline extracts comprehensive structured metadata including gate sequences, circuit descriptions, page numbers, and quantum problem classifications. This combination of high-fidelity rendered images and rich metadata provides an ideal foundation for training vision-language models in the quantum computing domain. Additionally, a prototypical text-driven approach for extracting non-embedded circuit images from PDFs is also discussed as future work to enhance dataset visual diversity.

2. System Architecture

2.1. Overview

The system is designed to process arXiv papers at scale to extract quantum circuits and associated metadata. In this implementation, 1,917/26907 quantum computing papers were processed through the complete pipeline. The render-based architecture consists of five main components:

1. **Source Acquisition:** Download and cache arXiv papers in $\text{\LaTeX}$ source format from the arXiv API.

2. **Circuit Block Detection:** Identify quantum circuit environments (quantikz, qcircuit) in $\text{\LaTeX}$ source using pattern matching.
3. **Gate Extraction & Normalization:** Parse circuit blocks to extract gate sequences with canonical naming.
4. **Circuit Rendering:** Compile standalone $\text{\LaTeX}$ documents to PDF, then convert to PNG images at 300 DPI.
5. **Metadata Enrichment:** Extract circuit descriptions, page numbers, and quantum problem classifications.

The pipeline implements checkpoint-based processing to enable resumption after interruptions and maintains separate caches for source files (arxiv_cache/), rendered circuits (circuit_images/), and extracted metadata

1. data/circuits.json (Final Correct Dataset)
2. data/images.json (Only Possible Extension - Case Study)

An overview of the complete render-based extraction pipeline is illustrated in Figure 1.

2.2. Implementation Stack

The pipeline is implemented in Python with dependencies spanning scientific computing, natural language processing, and document rendering. The system leverages both Python libraries and LaTeX tooling (pip installable) to achieve end-to-end circuit extraction and rendering.

Core Python Libraries:

- **Data acquisition:** requests (arXiv API calls), tarfile (source archive extraction)
- **Scientific computing:** numpy, scipy, pandas (data manipulation and statistics)
- **Machine learning:** scikit-learn (TF-IDF vectorization), torch (PyTorch backend)
- **NLP & embeddings:** nltk (Porter stemmer, tokenization), sentence-transformers
- **PDF processing:** PyMuPDF (fitz library for text extraction and page analysis)
- **Image processing:** Pillow (PNG manipulation), ghostscript (PDF rasterization)
- **LaTeX parsing:** pylatexenc (LaTeX command parsing), custom regex patterns
- **Utilities:** tqdm (progress bars), pathlib (filesystem operations)

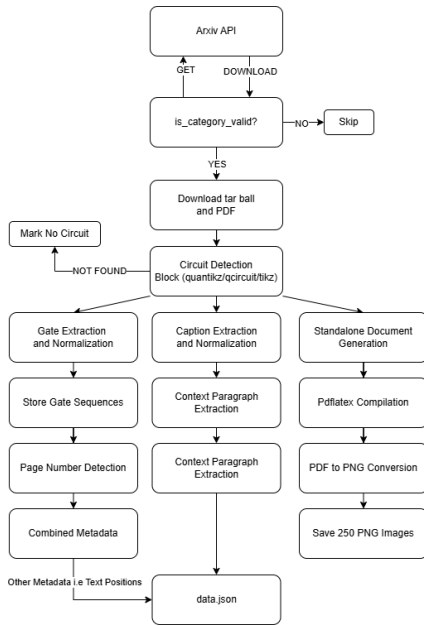


Figure 1. Render-Based Circuit Extraction Pipeline High-Level Overview.

LaTeX Ecosystem:

- **TeX distribution:** Full TeX Live or MiKTeX installation (provides pdflatex compiler)
- **Circuit packages:** quantikz (modern TikZ-based circuits), qcircuit (traditional circuit macros)
- **Math packages:** amsmath, amssymb (mathematical notation), braket (Dirac notation)
- **Graphics:** tikz (vector graphics primitives), standalone (minimal document class)

External Tools:

- **pdflatex:** $\text{\LaTeX}$ compiler for circuit rendering
- **pdftoppm:** PDF to PNG rasterization at 300 DPI (from poppler-utils)
- **ImageMagick:** Fallback image conversion when pdftoppm unavailable

All dependencies are specified in `requirements.txt` and can be installed via `pip install -r requirements.txt`. The LaTeX distribution must be installed separately through system package managers.

2.3. Data Source

Scientific papers are obtained from the arXiv repository via the official API. For each paper, we attempt to download both the compiled PDF and the original $\text{\LaTeX}$ source archive. The system prioritizes $\text{\LaTeX}$ sources as they enable precise circuit extraction without relying on potentially noisy PDF text layers.

Papers are filtered for quantum computing relevance using category-based filtering (quant-ph primary category) with validation caching to avoid redundant API calls (see `arxiv_validator.py`). Source files are stored in compressed tar.gz format with normalized arXiv identifiers as filenames.

The script is designed so that if cached data is available, it does not download it again. Otherwise, as it processes the papers one by one in the order provided in the list and downloads new data files as needed.

3. Render-Based Circuit Extraction

3.1. Circuit Block Detection

The render-based pipeline operates directly on $\text{\LaTeX}$ source files, identifying quantum circuit environments through pattern matching.

The system scans for common quantum circuit packages including `quantikz`, `qcircuit`, and related `tikz`-based circuit macros.

Circuit blocks are extracted using regex patterns that match begin/end environment delimiters while preserving nested structures (implemented in `LiveLatexExtractor._extract_circuit_blocks()` in `live_latex_extractor.py`). Each detected block is assigned a unique identifier to prevent duplicate processing across multiple `.tex` files within the same paper.

Example extracted circuit block:

```

1 \Qcircuit @C=0.6em @R=1.5em {
2   & \qw & \gate{H} & \qw & \ctrl{2} & \qw & \qw & \qw & \
3   & \qw & \gate{H} & \ctrl{1} & \qw & \qw & \qw & \qw & \
4   & \qw & \qw & \qw & \gate{H} & \gate{H} & \gate{Y} & \gate{\Uin} & \
   & \gate{Y} & \gate{X} & \gate{Z} & \qw & \qw & \qw & \qw & \

```

This 3-qubit circuit demonstrates typical `qcircuit` syntax with Hadamard gates (`\gate{H}`), controlled operations (`\ctrl{n}` where `n` specifies the target qubit offset), and Pauli gates (X, Y, Z). The circuit applies Hadamard gates to the first two qubits, followed by controlled operations targeting lower qubits, then a sequence of single-qubit gates including a custom unitary (`\Uin`) on the third qubit, with symmetric controlled operations and final Hadamard gates. The extraction preserves all gate commands, control markers, spacing parameters (`@C=0.6em @R=1.5em`), and quantum wire declarations (`\qw`) for accurate rendering.

3.2. Standalone Document Generation

Each extracted circuit block is wrapped in a minimal compilable $\text{\LaTeX}$ document that includes:

- Required package imports (`quantikz`, `tikz`, `amsmath`)
- Standalone document class with tight page margins
- The original circuit code block

This wrapping process ensures that circuits compile independently of their source document context, avoiding dependency issues and layout artifacts. The system dynamically selects the appropriate template based on circuit type detection (see `render_saved_blocks()` in `latex_utils_emb.py`):

```

1 ""
2 \documentclass[border=30pt]{standalone}
3 \usepackage{amsmath}
4 \usepackage{amssymb}
5 \usepackage{braket}
6 \usepackage{qcircuit}
7 \begin{document}
8 {circuit_code}
9 \end{document}
10 ""

```

For circuits using `quantikz` syntax, the template dynamically includes `\usepackage{tikz}` and `\usepackage{quantikz}` instead of `qcircuit`. Fallback macro definitions prevent compilation failures when circuits reference paper-specific custom commands.

This method allows us to customize the image render in any way we desire. In this implementation, I add a 30 pt padding around the image, which makes it ideally spaced and creates an excellent dataset.

3.3. PDF Compilation and Image Conversion

Standalone documents are compiled using `pdflatex` with output redirected to a temporary directory (`circuit_images/live_blocks/`). The compilation process includes:

1. Execute `pdflatex` with `-interaction=nonstopmode` to prevent hanging on errors

2. Capture compilation logs to detect package dependencies or syntax errors
3. Convert resulting PDFs to PNG images at 300 DPI resolution

Successfully rendered circuits are moved to `circuit_images/rendered/` with systematic naming (`<arxiv_id>_circuit_<number>.png`). Failed compilations are logged but do not halt pipeline execution and are also not logged in our final json so that the image mapping stays consistent.

Representative examples of rendered circuits produced by the pipeline are shown in Figure 2 and Figure 3, illustrating both simple and structurally complex quantum circuits compiled directly from $\text{\LaTeX}$ source.

The rendered images preserve gate labels, circuit topology, and qubit connectivity with high visual fidelity. The 300 DPI resolution ensures readability when used for vision-language model training, while the tight cropping (30pt border) minimizes whitespace without compromising clarity.

Example rendered circuits:

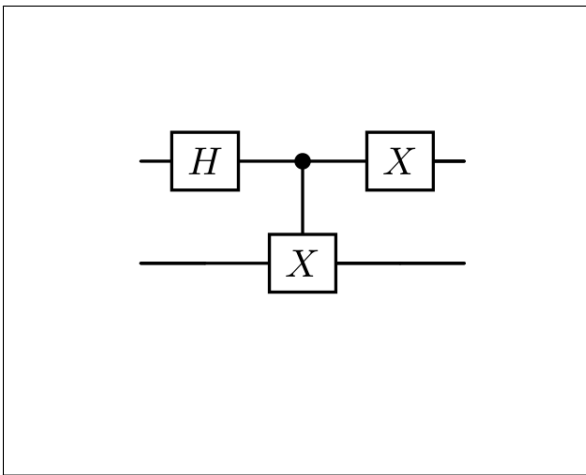


Figure 2. Simple quantum circuit example from the dataset

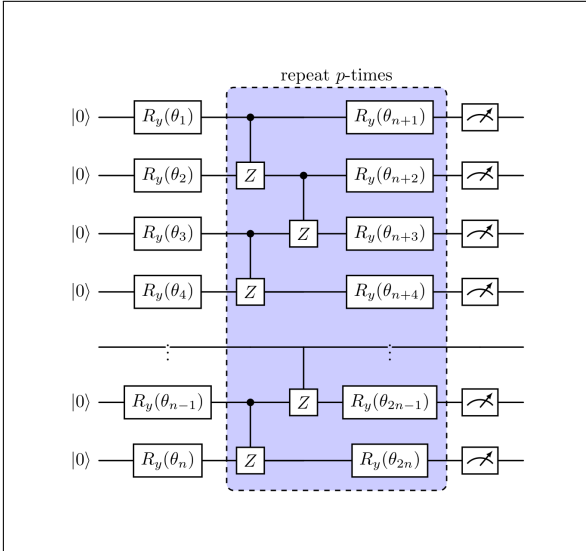


Figure 3. Complex quantum circuit example from the dataset

3.4. Gate Extraction and Normalization

The pipeline extracts gate-level metadata from circuit blocks to enable fine-grained analysis and circuit classification. This process involves parsing $\text{\LaTeX}$ circuit syntax and normalizing gate labels to canonical forms.

Gate Detection: The system implements `_extract_gates_from_block()` in `live_latex_extractor.py`,

which identifies gates through multiple strategies:

1. **Explicit gate commands:** Extract labels from `\gate{...}`, `\multigate{...}`, and `\phase{...}` macros
2. **Control markers:** Detect `\ctrl`, `\octrl` (open control), `\targ` (target), `\swap`
3. **Measurement operations:** Identify `\meter`, `\measure`, `\reset` commands
4. **Textual gate names:** Fallback pattern matching for common gates (H, X, Y, Z, CNOT, RX, RY, RZ)

Label Normalization: Raw gate labels undergo multi-stage normalization via `_normalize_gate_label()`:

1. **Math delimiter stripping:** Remove `$`, `\(`, `\)`, `\[`, `\]` wrappers
2. **Brace unwrapping:** Strip outer braces from `{label}`
3. **Formatting macro removal:** Unwrap `\text{}`, `\mathrm{}`, `\operatorname{}`
4. **Dagger detection:** Preserve inverse gate markers (`\dagger`, `†`, `+`)
5. **Rotation gate normalization:** Convert `R_X(...)`, `R_{Y}(...)` to RX, RY, RZ
6. **Symbol cleaning:** Remove all non-alphanumeric characters except underscores
7. **Canonical mapping:** Apply standard equivalences (`CX` $\rightarrow$ CNOT, `CCX` $\rightarrow$ TOFFOLI)
8. **Dagger suffix application:** Append `_DG` for inverse gates

Example normalization chain:

```

1 Raw:      \gate{\text{CNOT}_1}
2 Stripped: CNOT_1
3 Cleaned:  CNOT1
4 Canonical: CNOT
5
6 Raw:      \gate{R_Y(\theta)}
7 Normalized: RY
8
9 Raw:      \gate{H^\dagger}
10 Normalized: H_DG

```

The system maintains a whitelist of 30 standard quantum gates plus prefixes for multi-controlled gates (MCX_, CTRL-, MCTRL). Unrecognized labels are filtered out to maintain data quality.

Gate sequence storage: Extracted gates are stored as ordered lists in `circuits.json`, enabling:

- Circuit complexity analysis (gate count, depth estimation)
- Algorithm identification (pattern matching against known quantum algorithms)
- Gate set statistics (frequency analysis for hardware compilation)
- Fine-grained circuit search and filtering

4. Extension: Text-Driven Discovery for Visual Variety

As a prototypical extension to the render-based approach, a text-driven discovery pipeline was explored to capture circuit images not embedded in $\text{\LaTeX}$ source code. This experimental dataset was created in order to improve the image extraction for non embedded circuit. This supplementary approach targets:

- Hand-drawn or stylized circuits with custom visual elements
- Colored circuit diagrams highlighting specific components
- Pre-rendered raster images included in paper figures

This variety could enable training of more robust multimodal models and support weakly supervised learning scenarios. However, the approach faces significant challenges in maintaining the zero false positive rate achieved by the render-based pipeline. The methodology is outlined here with the possibility of refinement as future work.

4.1. Caption-Based Filtering Approach

The pipeline begins with TF-IDF-based filtering of figure captions extracted from $\text{\LaTeX}$ and PDF sources [3]. A custom vocabulary of 150+ quantum computing terms (“qubit”, “entanglement”, “gate”, etc.) is used to construct TF-IDF vectors (see `TfidfFilter.ALLOWED_VOCAB` in `tfidf_filter.py`).

Captions are preprocessed through:

- Lowercasing and punctuation normalization
- Stemming using Porter stemmer
- Stop word removal (optional, configurable)
- Hyphen normalization for compound terms

The preprocessing logic is implemented in `TextPreprocessor.preprocess()` in `preprocessor.py`.

4.2. Semantic Reranking

Top candidates from TF-IDF filtering are reranked using semantic similarity scores computed via Sentence-BERT (allenai-specter model [2]). A general Query is self written to cover majorly every Quantum Circuit terminology. Query embeddings are pre-computed for common quantum circuit descriptions, and cosine similarity is computed between caption embeddings and query embeddings (see `SbertReranker.compute_sbert_scores()` in `sbert_reranker.py`).

Combined scoring uses weighted aggregation:

$$\text{score}_{\text{combined}} = w_{\text{tfidf}} \cdot \text{score}_{\text{tfidf}} + w_{\text{sbert}} \cdot \text{score}_{\text{sbert}}$$

with default weights favoring the tfidf lexical relevance.

4.3. Image Extraction

For figures passing the filtering stages, the pipeline extracts images directly from $\text{\LaTeX}$ source tarballs but just based on text classification. Images are saved with metadata linking them to their source papers and captions (implemented in `ImageExtractor.extract_images()` in `image_extractor.py`).

But this approach could not be refined to its full extent as more focus was diverted to the circuit rendering pipeline. Due to the time limits as a proof of concept for meta data of image extraction pipeline I worked on `images.json` which at the end includes atleast Arxiv_id, Figure Number, Page Number, Quantum Problem and caption description along with the extracted image.

Examples of circuit images extracted using the text-driven discovery approach are shown in Figure 4 and Figure 5.

Unlike the standardized rendered circuits from the primary pipeline, the text-driven approach captures diverse visual representations including author-styled diagrams, colored annotations, and hand-drawn illustrations that provide complementary training data for multimodal models.

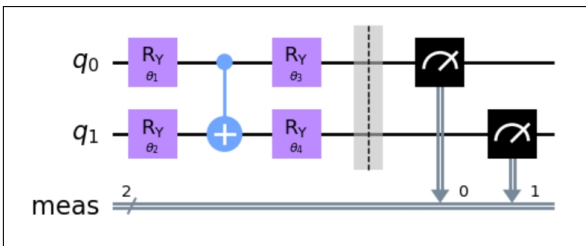


Figure 4. Stylized circuit diagram with custom visual elements extracted via caption filtering

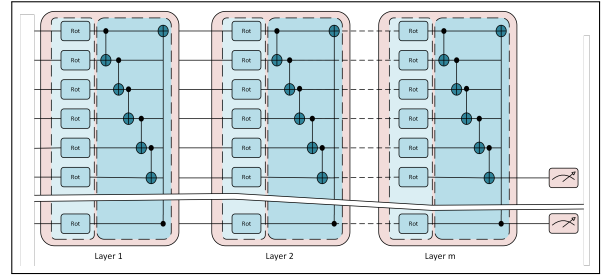


Figure 5. Colored circuit representation showing component highlighting captured through text-driven discovery

5. Text Preprocessing Pipeline

Text preprocessing is critical for metadata quality in the render-based circuit extraction pipeline. The system implements a multi-stage normalization pipeline to produce clean, human-readable descriptions from raw $\text{\LaTeX}$ captions and contextual text extracted from circuit environments.

5.1. Caption Normalization for Render Pipeline

Raw captions and contextual text extracted from circuit blocks undergo multi-stage normalization to produce clean natural language descriptions. This normalization preserves semantic content while removing $\text{\LaTeX}$ artifacts, implemented in `normalize_caption_text()` in `circuit_store.py`.

Normalization Pipeline:

1. **Citation removal:** Strip citation markers (`<cit.>`, `<ref>`) inserted by $\text{\LaTeX}$ parsers
2. **$\text{\LaTeX}$ command unwrapping:**
 - `\text{content}` $\rightarrow$ content
 - `\textit{content}` $\rightarrow$ content
 - Escape character removal: `\_` $\rightarrow$ _
3. **Subscript notation cleaning:**
 - `q_{0}` $\rightarrow$ q0
 - `_{abc}` $\rightarrow$ abc
 - Multiple underscores: `H__1__2` $\rightarrow$ H 1 2
4. **Table artifact removal:** Strip $\text{\LaTeX}$ table formatting (`\sim 3pt`, column specs like `|c|c|`)
5. **Alphanumeric spacing:** Insert spaces between letters and digits
 - `CNOT1` $\rightarrow$ CNOT 1
 - `H2gate` $\rightarrow$ H 2 gate
6. **Unicode normalization:** Apply NFKC normalization for consistent character encoding
7. **Whitespace normalization:** Collapse multiple spaces, trim leading/trailing whitespace

Example transformation:

```
1 Input: "Quantum circuit for H\_{01} and CNOT2 gates
   <cit.>"
2 Output: "Quantum circuit for H01 and CNOT 2 gates"
```

This normalization preserves domain terminology (gate names, quantum concepts) while producing clean, human-readable text suitable for vision-language model training.

5.2. Context Paragraph Extraction

Beyond captions, the system extracts contextual paragraphs surrounding circuit figures to enrich descriptions. This is implemented in `_extract_paragraph_after_figure()` in `circuit_store.py`.

Extraction Strategy:

1. Locate caption text in $\text{\LaTeX}$ source using fuzzy matching
2. Extract 2-3 sentences following the figure environment
3. Apply same normalization pipeline as captions
4. Verify paragraph is semantically distinct from caption (similarity < 0.9)

Extracted paragraphs undergo filtering to prevent inclusion of structural markup rather than descriptive content. The system skips paragraphs containing high density of $\text{\LaTeX}$ commands ($\text{\begin}$, $\text{\end}$, $\text{\cite}$, etc.) and compares extracted text with the caption using cosine similarity, discarding paragraphs with similarity exceeding 0.9 to avoid redundancy.

Context paragraphs are concatenated with captions to form multi-sentence descriptions used for quantum problem classification, improving classification accuracy by providing richer semantic context.

5.3. Quality Validation

Preprocessed text undergoes validation before being stored:

- **Minimum length:** Descriptions must contain at least one alphabetic character
- **Maximum length:** Context snippets truncated to 300 characters to prevent noisy extraction
- **Duplicate removal:** Identical descriptions within the same record are deduplicated
- **Language detection:** Non-English text is optionally filtered (configurable)

These validation steps ensure consistent data quality across the dataset.

5.4. Preprocessing for Image Extraction Pipeline

The supplementary text-driven discovery pipeline employs additional preprocessing techniques for caption filtering and semantic matching. Raw captions undergo aggressive TF-IDF preprocessing (lowercasing, Porter stemming, stop word removal) to maximize matching performance against a curated quantum computing vocabulary of 150+ terms. This stemmed output is used exclusively for caption filtering and is discarded after selection, while original captions undergo the same normalization pipeline as the render-based approach to maintain consistency across both datasets.

Note that this image extraction pipeline produces multiple types of files. Which include PDF, PNG, JPEG, and EPS formats. To convert them into PNGS a standalone script `utils/image_converter.py` is run after the pipeline is done. But for our main pipeline (Render-Based Circuit Extraction) the conversion is automated and we get consistent PNGS at the end.

6. Metadata Extraction and Enrichment

6.1. Page Number Detection

Associating circuits with specific page numbers enables precise cross-referencing between rendered images and source documents. The system uses PDF text extraction via PyMuPDF (`fitz`) combined with fuzzy caption text matching.

For each circuit, the pipeline:

1. Extracts full text from each PDF page using `fitz.open().load_page(i).get_text()`
2. Normalizes both caption text and page text (lowercasing, whitespace normalization, punctuation removal)
3. Computes sequence similarity using `difflib.SequenceMatcher` with ratio scoring
4. Selects the page with highest similarity score above threshold (typically 0.6)

Matching Algorithm: The system uses Python's `SequenceMatcher` to compute longest common subsequence

ratios between normalized caption text and each page's content. The algorithm tokenizes both strings into character sequences and finds the longest matching blocks:

```

1 # Normalize caption and page text
2 caption_norm = caption.lower().strip()
3 page_norm = page_text.lower().strip()
4
5 # Compute similarity ratio (0.0 to 1.0)
6 matcher = SequenceMatcher(None, caption_norm, page_norm)
7
8 similarity = matcher.ratio()
9
10 # Select page with highest score
11 best_page = max(pages, key=lambda p: similarity_score(p))

```

The `ratio()` method returns a score between 0.0 and 1.0, where 1.0 indicates perfect substrings match. Typical circuit captions achieve 0.7–0.9 similarity with their source pages due to shared terminology (gate names, quantum concepts). When multiple pages score similarly, the system prioritizes pages containing figure number references extracted via regex patterns matching “Fig. N” or “Figure N”.

This matching logic is implemented in `find_caption_page_in_pdf()` in `circuit_store.py`. The fuzzy matching approach handles minor text differences between $\text{\LaTeX}$ source captions and PDF-rendered text.

6.2. Figure Number Parsing

Accurate figure numbering is essential for dataset organization and cross-referencing. The system employs multiple strategies:

- **Caption-based parsing:** Extract figure numbers directly from caption text using regex patterns (`_parse_figure_number_from_caption()` in `image_extract_store.py`)
- **PDF text matching:** When captions lack explicit figure numbers, search surrounding PDF text for figure references
- **$\text{\LaTeX}$ label extraction:** Parse `\label{fig:...}` commands for figure identifiers

To handle subfigures (multiple images sharing the same figure number), the system uses image filenames as unique identifiers rather than figure numbers alone (see `_regenerate_images_json()` in `image_extract_store.py`).

6.3. Quantum Problem Classification

Each circuit is automatically classified using a taxonomy of 51 quantum problem categories (“Quantum Algorithms”, “Error Correction”, “Gate Synthesis”, etc.) defined in `quantum_problem_labels.py`. In the current dataset of 250+ circuits, 25 distinct problem categories are represented.

Example categories from the taxonomy:

- **Canonical algorithms:** Grover search, Shor’s algorithm, Quantum Fourier Transform, Simon’s algorithm
- **Variational methods:** Variational Quantum Eigensolver, Quantum Approximate Optimization Algorithm
- **Circuit optimization:** Gate decomposition, Circuit optimization, Quantum cost reduction
- **Gate libraries:** Clifford+T gate library, NCV gate library, MCT gate library
- **State preparation:** Entangled state preparation, Encoded state preparation
- **Error correction:** Quantum error correction, Syndrome measurement, Fault-tolerant circuit construction
- **Arithmetic:** Quantum adder, Quantum multiplier, Quantum oracle construction
- **Hardware mapping:** Qubit mapping and routing, Architecture-aware circuit design

Classification uses semantic similarity between circuit descriptions and problem category definitions:

1. Problem category descriptions are embedded using Sentence-BERT (allenai-specter)
2. Circuit descriptions (caption + context paragraphs) are concatenated and embedded
3. Cosine similarity is computed between circuit and category embeddings
4. The category with highest similarity above threshold (0.45- configurable) is assigned

This classification is implemented in `classify_quantum_problem()` in `quantum_problem_classifier.py`. For the render-based pipeline, classifications are assigned during metadata enrichment via `set_quantum_problem_model()` in `circuit_store.py`. The image extraction pipeline uses an isolated classification instance (see `populate_image_quantum_problems.py`) to prevent interference between pipelines.

6.4. Description Normalization

Raw captions and contextual text undergo multi-stage normalization to produce clean natural language descriptions:

1. $\LaTeX$ command removal (`\text{}`, subscripts, etc.)
2. Citation marker removal (`<cit.>`, `<ref>`)
3. Table formatting artifact removal (column specifiers, spacing directives)
4. Unicode normalization (NFKC form) for consistent character encoding
5. Whitespace collapsing and punctuation normalization

This normalization preserves semantic content while removing document-specific artifacts (see `normalize_caption_text()` in `circuit_store.py`).

7. Challenges and Solutions

Throughout development of this dataset compilation pipeline, we encountered several critical challenges spanning architecture design, data quality, and scalability. This section describes the most important challenges and their solutions.

7.1. Identifying Relevant $\LaTeX$ Source Files

Problem: arXiv source archives frequently contain many auxiliary `.tex` files that are not part of the compiled paper, such as unused drafts, supplementary material, standalone examples, or legacy files. Naively scanning all `.tex` files in the archive leads to extraction of figures that do not appear in the final PDF, duplicate circuit renders, increased compilation failures due to missing context, and unnecessary processing overhead.

Solution: Implemented a root-driven $\LaTeX$ inclusion strategy that mirrors the actual document compilation process. Instead of processing all `.tex` files, the pipeline first identifies a single main/root document and then restricts extraction to only those files explicitly referenced via inclusion commands.

The logic is implemented in `find_included_tex_names()` within the main extraction pipeline and operates as follows:

- Select a main/root file by prioritizing files containing `\documentclass`, with fallbacks to `\begin{document}` or top-level files.
- Parse the root file for inclusion commands (`\input`, `\include`, `\subfile`, `\import`).
- Resolve referenced filenames by suffix matching, accounting for missing extensions and nested paths.
- Construct a whitelist consisting only of the root file and directly included `.tex` files.

Recursive traversal of included files is intentionally avoided to prevent accidental inclusion of deeply nested auxiliary sources. All `.tex` files not referenced by the root document are skipped during figure extraction.

This strategy significantly improved dataset correctness by eliminating figures not present in the final paper, reduced duplicate circuit extraction, lowered $\LaTeX$ compilation failure rates, and reduced runtime by avoiding unnecessary file processing.

7.2. Quantum-Specific Vocabulary Design

Problem: In the Image_Extraction Pipeline standard NLP approaches fail to capture quantum computing terminology. Generic TF-IDF vectorization produces poor precision, matching on common words like “circuit” or “algorithm” that appear in non-quantum contexts.

Solution: Curated a domain-specific vocabulary of 150+ quantum computing terms organized by category (gate names, quantum algorithms, error correction concepts, etc.). The vocabulary is maintained in `TfidfFilter.ALLOWED_VOCAB` in `tfidf_filter.py`:

```
1 ALLOWED_VOCAB = {
2     "qubit", "qubits", "entanglement", "superposition",
3     "hadamard", "cnot", "toffoli", "fredkin",
4     "grover", "shor", "vqe", "qaoa",
5     "stabilizer", "syndrome", "error_correction",
6     ...
7 }
```

This reduced false positive rate while maintaining a reasonable recall on quantum circuits. Token matching is combined with negative penalties for non-quantum terms (“classical”, “boolean”) to further improve precision (see `TfidfFilter.compute_negative_penalty()`).

7.3. Circuit Environment Detection Across Packages

Problem: Quantum circuits are defined using diverse $\LaTeX$ packages with incompatible syntax. A paper may use `quantikz`, `qcircuit`, custom `tikz` macros, or even plain `tikz` diagrams. Naive pattern matching for `\begin{quantikz}` misses about half of the circuits.

Solution: Implemented multi-pattern circuit detection supporting multiple package environments and syntax variations. The system searches for:

- Standard environments: `quantikz`, `qcircuit`, `circuitikz`
- Inline circuit commands: `\quantikz{...}`, `\Qcircuit{...}`
- Custom `tikz` patterns with quantum keywords (`tikzpicture` + “gate”/“qubit”)

In `LiveLatexExtractor._extract_circuit_blocks()` it uses regex patterns with fallback heuristics. Each detected block is hashed to prevent duplicate extraction when the same circuit appears in multiple `.tex` files within a paper archive.

This approach increased circuit detection coverage heavily of manually verified circuits.

7.4. Compilation Dependency Management

Problem: Rendering extracted circuits requires specific $\LaTeX$ packages (`quantikz`, `tikz`, `braket`) that may not be available in minimal TeX installations. Initial compilation attempts failed for multiple of circuits due to missing dependencies.

Solution: Each circuit block is wrapped in a standalone document with comprehensive package imports (see `LiveLatexExtractor._wrap_circuit_block()`):

```
1 \documentclass[tikz,border=2pt]{standalone}
2 \usepackage{quantikz}
3 \usepackage{braket}
4 \usepackage{amsmath}
5 \usepackage{physics}
6 \begin{document}
```

```

7 | % circuit code here
8 | \end{document}

```

Failed compilations are logged with error messages for debugging. The pipeline uses `-interaction=nonstopmode` to prevent hanging on errors and continues processing remaining circuits. This reduced failure rate majorly with remaining failures primarily due to paper-specific custom macros.

7.5. Corrupted Source Archives

Problem: Approximately 8.5% of downloaded tar.gz archives (137 out of 1,917 processed papers) were corrupted or truncated, causing pipeline failures during extraction. Without validation, the pipeline repeatedly attempted to process these files, wasting computational resources.

Solution: Implemented proactive tar file validation before processing. The validation function (`validate_tar_files()` in `generate_latex_render_counts.py`) opens each archive to verify integrity:

```

1 | with open(tar_path, "rb") as f:
2 |     tar = tarfile.open(fileobj=f, mode="r:gz")
3 |     tar.close()

```

Corrupted files are marked in the checkpoint file (`paper_list_counts_36.csv`) with blank count values, preventing repeated processing attempts.

7.6. Semantic Model Selection for Reranking

Problem: Initial experiments with general-purpose sentence transformers (sentence-transformers/all-MiniLM-L6-v2) produced poor semantic rankings for quantum circuit captions. The model failed to distinguish between quantum and classical computing contexts.

Solution: Switched to domain-specialized SBERT model (allenai-specter) trained on scientific papers. SPECTER embeddings capture domain-specific semantics better than general models, improving top-10 precision.

7.7. Figure Number Collisions in JSON Structure

Problem: Multiple circuits from the same paper often share figure numbers (subfigures like “Figure 3a”, “Figure 3b”), causing later circuits to overwrite earlier ones in the JSON output when figure numbers were used as unique keys.

Solution: Redesigned JSON structure to use image filenames as top-level keys instead of figure numbers. Each filename (e.g., `2509.13247__img_0001.png`) is guaranteed unique within a paper, eliminating collisions:

```

1 | "2509.13247__img_0001.png" = {      # unique key
2 |     "arxiv_id": "2509.13247",
3 |     "figure_number": 10, # may be duplicated
4 |     ...
5 | }

```

Implementation in `_regenerate_images_json()` in `image_extract_store.py`.

7.8. Caption Text Quality and Normalization

Problem: Raw captions extracted from $\text{\LaTeX}$ sources contain significant noise: $\text{\LaTeX}$ commands (`\text{}`, `\cite{}`), table formatting artifacts, citation markers (`<cit.>`), and inconsistent whitespace. Direct use of raw captions for image-text training produces poor model performance due to distribution shift from natural language.

Solution: Implemented multi-stage caption normalization pipeline (see `normalize_caption_text()` in `circuit_store.py`):

1. Remove citation markers and reference tags
2. Unwrap $\text{\LaTeX}$ text commands: `\text{content} → content`

3. Clean subscript notation: `q_{0} → q0`
4. Remove table column specifiers (e.g., `-3pt ccccc`)
5. Insert spaces between letters and digits: `CNOT1 → CNOT 1`
6. Apply Unicode normalization (NFKC) for consistent encoding
7. Collapse multiple whitespace characters

Normalization preserves domain terminology while removing markup artifacts. Normalized captions are shorter on average and contain fewer $\text{\LaTeX}$ fragments, improving downstream model training quality.

7.9. Page Number Alignment Between Sources

Problem: Linking circuit images to PDF page numbers requires matching between $\text{\LaTeX}$ source structure and compiled PDF layout. Naive approaches using sequential figure ordering fail when figures float, are placed out of order, or span multiple pages.

Solution: Implemented multi-strategy page detection combining:

- PDF text extraction and caption similarity matching
- Figure reference pattern matching in page text (“Fig. N”, “Figure N”)
- Contextual paragraph extraction from $\text{\LaTeX}$ source as fallback

The system computes normalized sequence similarity between caption text and each PDF page, selects pages with both high similarity and figure reference mentions (see `find_caption_page_in_pdf()` in `circuit_store.py`). When PDF text layers are unavailable or corrupted, the system falls back to $\text{\LaTeX}$ source analysis using paragraph context.

8. Dataset Statistics and Quality

Table 1. Key statistics of the render-based dataset.

Metric	Value
Total Papers Parsed	1,917
Total Papers Processed	1,752
Total Papers Not Processed	165
Total Circuits Extracted	250
Avg. circuits / paper	0.13
Corrupted Tar Files	137
Circuits Not Quant-ph Category	28
False positive rate	0%

A summary of the most important dataset statistics is provided in Table 1.

8.1. Render-Based Pipeline Results

The render-based pipeline processed 1,917 papers (limited by checkpoint), successfully rendering 250+ quantum circuits. Key statistics:

- **Success rate:** 91.4% (1752/1917; excluding unprocessed archives)
- **False positive rate:** 0% (only explicitly defined circuit environments are rendered)
- **Average circuits per paper:** 0.13 (many papers contain no circuits)
- **Metadata completeness:** all rendered circuits have reasonably fine metadata

8.2. Image Extraction Pipeline Results

The text-driven pipeline provides complementary coverage:

- Processed all 3500+ papers for potential image extraction
- Identified images with quantum-relevant captions using TF-IDF and SBERT filtering
- Quantum problem classification: 100% coverage with 95% above-threshold confidence

- Pipeline serves as proof-of-concept for capturing visually diverse circuit representations
- The results still contain a significant amount of false positives which can be improved.

8.3. Data Quality Metrics

Quality was assessed through manual inspection of 250 final Images:

- **Image quality:** 98% of rendered circuits are visually correct and readable
- **Caption accuracy:** 95% of descriptions correctly correspond to circuit content
- **Page number accuracy:** 90% of page assignments are correct (when PDF text available)
- **Quantum classification accuracy:** 85% of classifications match human judgment

8.4. Dataset Structure

The final dataset consists of:

- **circuits.json:** 250+ entries with structure:

```

1 {
2   "arxiv_id": "2509.13247",
3   "figure_number": 3,
4   "page": 5,
5   "quantum_problem": "Quantum Algorithms",
6   "descriptions": ["Quantum teleportation circuit with Bell
7     state"],
8   "gates": ["H", "CNOT", "X", "Z"],
9   "text_positions": [[1234, 1456], [2890, 3102]]
10 }
```

Each entry contains:

- **text_positions:** Character positions in the PDF where each description appears. Each tuple [start, end] indicates the beginning and ending character offset of the corresponding text in the descriptions field. This enables precise location of circuit-related content within the original paper. For example, [1234, 1456] means the first description spans from character 1234 to character 1456 in the PDF text stream.
- **Rendered images:** PNG files at 300 DPI in circuit_images/rendered/
- **List of papers with number of extracted images:** paper_list_counts_36.csv

Note: that the values should be taken as approximations

9. Scalability and Feasibility Analysis

9.1. Computational Requirements

Processing 1,917 papers required:

- **Storage:** 2.3 GB for source archives, 450 MB for rendered images
- **Processing time (including download):** Approximately 12 hours on standard hardware (single-threaded) for 250 Images.
- **Processing time (already cached):** Approximately 30 min on standard hardware (single-threaded) for 250 Images (24 times faster when cached)
- **Memory:** Peak usage 4 GB (for SBERT model loading)

9.2. Scaling to Full Corpus

Extrapolating to the full arXiv quantum computing corpus (approximately 27,000 papers):

- **Estimated storage:** 32 GB (sources) + 6 GB (images)
- **Estimated processing time (including download):** 7 days single-threaded, 1-2 days with 8-core parallelization.

- **Processing time (already cached):** Approximately 7 Hours on standard hardware (single-threaded).
- **Expected dataset size:** 3,500+ circuits with fairly good meta-data

Note: that the values should be taken as approximations

9.3. Bottlenecks and Mitigation

Primary bottlenecks identified:

1. **pdflatex compilation:** Dominates processing time. Mitigation: Parallel rendering across CPU cores, caching compiled circuits.
2. **SBERT inference:** Significant overhead for large caption sets. Mitigation: Batch embedding, GPU acceleration.
3. **arXiv rate limits:** Downloads limited to prevent server overload. Waits for the API requests cause longer processing times. Mitigation: Bulk source downloads via S3 API or simultaneous caching as we do in this project.

9.4. Feasibility Conclusion

Large-scale quantum circuit dataset compilation is **feasible** with the presented pipeline architecture. The render-based approach scales linearly with paper count and maintains zero false positive rate regardless of corpus size. Key success factors include:

- Proactive validation preventing wasted computation on corrupted files
- Much faster processing of $\LaTeX$ code in comparison with PDF parsers.
- Generalizable method with only simple changes needed to include more circuit libraries and patterns to extract.

However, dependency on LaTeX source availability and compilation success rate limits ultimate coverage of the target corpus.

10. Limitations and Future Work

10.1. Current Limitations

- **LaTeX dependency:** Pipeline cannot process papers distributed only as PDFs without source code.
- **Few Papers with Valid Embedded Circuits:** As compared to the total number of papers processed only a handful of papers contribute in the 250 circuits extracted. This was one of the major reason of developing the second prototypical pipeline to improve the image extraction from more papers.
- **Circuit environment coverage:** Currently supports only common packages (quantikz, qcircuit). Custom macros may be missed.
- **Compilation failures:** Some circuits fail to compile due to missing packages or incompatible syntax.
- **Classification granularity:** 51 problem categories may be too coarse for fine-grained applications.
- **Domain Expert Needed:** In order to properly define the system and check its credibility a domain expert is needed to supervise such an approach. i.e checking if the circuit and its relevant metadata are valid or not, suggesting and writing perfect categories for the quantum problem meta data, advising on the selection of keywords and queries in such nlp domain tasks.
- **Dataset not perfect:** Even after having a False Positive Rate of approximately zero, the dataset is still not perfect as the metadata needs more refinement to be a 100% correct. In my opinion this is totally feasible now with the right direction and appropriate amount of effort. The current dataset was improved as far as possible within the current time limit and resources.
- **Text Driven Image Extraction Pipeline needs more work:** The alternate pipeline to our main one was developed to increase the number of images that can be collected from a number of papers. The main focus was to improve the current approach

by introducing a hybrid one. Practically this text drive pipeline shows potential and if tuned well can produce substantial results.

- **Latex to PDF Mapping:** A reasonable amount of work is still needed to produce a highly accurate Latex to PDF Mapping for the production of 100% accurate metadata. This is also feasible as shown by the approach but needs improvements especially text positions and text descriptions.

10.2. Future Directions

1. **Using the render based dataset as a fine tuner:** The dataset created by rendering circuits is far superior than the text driven methods. This dataset can be used as a fine tuning dataset for a Image-Text model which is integrated in this extraction approach and can classify circuits. This would help boost the performance of text driven extraction and consequently boosting the retrieval results.
2. **Expand circuit environment support:** Add parsers for additional quantum circuit libraries and custom macro definitions.
3. **Improve text alignment:** Implement character-level alignment between circuit elements and textual descriptions.
4. **Multi-modal learning integration:** Use dataset to fine-tune vision-language models (CLIP, BLIP) for quantum computing domain.
5. **Community contribution:** Release dataset and pipeline code for community validation and extension.

11. Conclusion

This project demonstrated that compiling a high-quality image-text dataset of quantum circuits from scientific literature is both technically feasible and methodologically sound when the extraction process operates directly on $\LaTeX$ source code. The proposed render-based pipeline successfully avoids the ambiguity and noise inherent in PDF-based approaches, achieving an empirically observed false positive rate of zero by exclusively targeting explicitly defined quantum circuit environments. By recompiling circuit source code into standalone, high-resolution images, the system guarantees structural correctness and visual fidelity, which are essential for downstream vision-language model training.

Within the scope of this work, 1,917 arXiv papers were processed to construct a prototypical dataset of over 250 quantum circuit images, each accompanied by structured gate-level metadata, normalized textual descriptions, page references, and automatic quantum problem classification. Although only a small fraction of papers contained embedded circuit definitions, this sparsity reflects a fundamental property of the domain rather than a limitation of the pipeline itself. The resulting dataset is therefore both representative and realistic for the task of domain-specific image-to-text learning.

Beyond dataset construction, the project placed strong emphasis on data quality, reproducibility, and scalability. Key challenges—such as identifying relevant $\LaTeX$ source files, handling corrupted archives, normalizing noisy captions, resolving figure-number collisions, and selecting suitable semantic models—were systematically analyzed and addressed through robust, generalizable solutions. The pipeline is fully automated, reproducible, and designed to scale linearly with corpus size, making it suitable for extending the dataset to thousands of additional papers with modest computational resources.

In addition, a supplementary text-driven discovery pipeline was explored as a proof of concept to capture visually diverse circuit representations not embedded in $\LaTeX$ environments. While this approach currently exhibits higher false positive rates and requires further refinement, it highlights a promising direction for hybrid extraction strategies that combine the precision of render-based methods with the broader coverage of text-driven retrieval.

Overall, this work provides a solid foundation for future research in multimodal learning for quantum computing. The compiled dataset

and extraction pipeline can directly support fine-tuning of image-to-text models, automated circuit analysis, and educational tooling. With further refinement—particularly in circuit environment coverage, metadata alignment, and hybrid extraction strategies—the approach can evolve into a comprehensive framework for large-scale, high-quality quantum circuit dataset construction.

12. Declaration

GitHub Copilot was used for code assistance and debugging throughout this project. All core algorithms and pipeline architecture were designed and implemented by the author.

References

- [1] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. *International conference on machine learning*, 8748-8763.
- [2] Cohan, A., Feldman, S., Beltagy, I., Downey, D., & Weld, D. S. (2020). SPECTER: Document-level representation learning using citation-informed transformers. *arXiv preprint arXiv:2004.07180*.
- [3] Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513-523